THOMAS MORE

LINUX WEB & NETWORK SERVICES

db-k8s-stack - Repository Documentation

2025 - 2026

Inhoud

Executive Summary 5

1. REPOSITORY OVERVIEW 6

1.1. Purpose and Scope 6

1.2. Core Technologies 6

2. REPOSITORY STRUCTURE 8

3. ARCHITECTURE OVERVIEW 9

3.1. High-Level Architecture 9

3.2. Kubernetes Namespaces 9

3.3. Cluster Topology 10

3.4. Port Map 10

4. COMPONENT ANALYSIS 11

4.1. Frontend (`frontend/`) 11

4.2. API (`api/`) 13

4.3. Database (`k8s/postgres/postgres.yaml`) 17

4.4. nginx Ingress (`k8s/ingress/ingress.yaml`) 19

4.5. Cloudflare Tunnel (`k8s/cloudflared/deployment.yaml`) 21

4.6. Monitoring Stack (`k8s/monitoring/`) 22

4.7. ArgoCD (`helm/argocd-values.yaml`, `k8s/argocd/application.yaml`) 24

5. RUNTIME AND REQUEST FLOWS 26

5.1. Application Start Sequence (fresh `vagrant up`) 26

5.2. HTTP Request Flow (browser → database) 27

5.3. Name Update Flow 28

5.4. Auto Layout Refresh Flow 29

5.5. GitOps Sync Flow 30

6. CONFIGURATION AND ENVIRONMENT VARIABLES 31

6.1. Application Environment Variables 31

6.2. Kubernetes ConfigMap (`k8s/configmap.yaml`) 32

6.3. Kubernetes Secret (`k8s/secret.example.yaml`) 32

6.4. `.env.example` 33

6.5. Configuration Risks and Gotchas 33

7. API AND INTERFACES 34

7.1. REST API Reference 34

7.2. Inter-Service Communication 36

7.3. Authentication and Security 36

8. DATABASE AND PERSISTENCE 38

8.1. PostgreSQL 38

8.2. Schema 38
8.3. Read/Write Patterns 38
8.4. Persistence Model 39

9. BUILD, RUN, TEST, AND DEPLOY 40
9.1. Building Images 40
9.2. Local Development (Docker Compose) 40
9.3. Running the Full Smoke Test 41
9.4. Deploying to Kubernetes 41
9.5. Cluster Lifecycle Commands 42
9.6. Updating the Application Name 42

10. KUBERNETES AND INFRASTRUCTURE 43
10.1. Kubernetes Version and Installation 43
10.2. Network Architecture 43
10.3. CNI — Flannel 43
10.4. Helm Releases 44
10.5. Resource Limits Summary 44

11. MONITORING, LOGGING, AND OPERATIONS 45
11.1. Prometheus 45
11.2. Grafana 46
11.3. Logging 47
11.4. Health Checks and Restarts 48
11.5. Common Failure Points 48

12. KEY SCRIPTS AND AUTOMATION 49
12.1. `vagrant/provision-common.sh` 49
12.2. `vagrant/provision-master.sh` 49
12.3. `vagrant/provision-worker.sh` 50
12.4. `deploy-k8s.sh` 50
12.5. `vagrant/scripts/update_name.sh` 50
12.6. `package.sh` 50

13. RISKS, OBSERVATIONS, AND IMPROVEMENT OPPORTUNITIES 51
13.1. Confirmed Issues 51
13.2. Observations / Inferred Risks 52
13.3. Improvement Opportunities 52

14. APPENDIX A: IMPORTANT FILES 54

15. APPENDIX B: COMMANDS REFERENCE 57
15.1. Cluster Management 57
15.2. Kubernetes — Cluster Inspection 57
15.3. Kubernetes — Application Operations 57
15.4. Kubernetes — ArgoCD 58
15.5. Kubernetes — Port Forwarding 58

15.6. Build and Deploy 59
15.7. Testing API Endpoints 59
15.8. Verifying Load Balancing (PowerShell) 59

16. APPENDIX C: GLOSSARY 60

17. APPENDIX D: PYTHON DEPENDENCIES 62

18. APPENDIX E: URLS & ENDPOINTS REFERENCE 63
18.1. Public URLs (via Cloudflare Tunnel) 63
18.2. Local Access (Vagrant Port Forwarding) 64
18.3. NodePorts (direct on cp1: 192.168.56.10) 65
18.4. API Endpoints 65
18.5. Internal Cluster DNS 66
18.6. Prometheus Scrape Targets 66

19. APPENDIX F: CLOUDFLARE TUNNEL SETUP 68
19.1. Prerequisites 68
19.2. Create the Tunnel 68
19.3. Add Public Hostnames (DNS entries) 69
19.4. TLS 71
19.5. Deploy the Tunnel Pod 71
19.6. Verify End-to-End 71

Executive Summary

db-k8s-stack is a production-minded three-tier web application deployed on a self-provisioned kubeadm Kubernetes cluster. The project demonstrates a complete DevOps workflow: container image authoring, Kubernetes manifest management, automated cluster provisioning with Vagrant and VirtualBox, public HTTPS exposure via a Cloudflare Tunnel, full observability with Prometheus and Grafana, and continuous GitOps deployment using ArgoCD.

The application stack consists of an Apache HTTPD frontend, a FastAPI backend, and a PostgreSQL database. It was built as a university assignment for the Thomas More Linux Web & Network Services course and is designed to achieve maximum scoring across all assignment criteria.

Key characteristics:

- Single command cluster creation: `vagrant up` builds a three-node kubeadm cluster from scratch
- No public IP required: Cloudflare Tunnel provides public HTTPS access
- Fully automated: provisioning scripts install the entire stack without manual steps
- GitOps: ArgoCD continuously reconciles the cluster state with the GitHub repository
- Observability: Prometheus, Grafana, kube-state-metrics, and node-exporter pre-installed

The main repository is located at: https://github.com/dabeastnet/db-k8s-stack

---

1. Repository Overview

1.1. Purpose and Scope

The repository implements a full application lifecycle for a three-tier web application in Kubernetes:

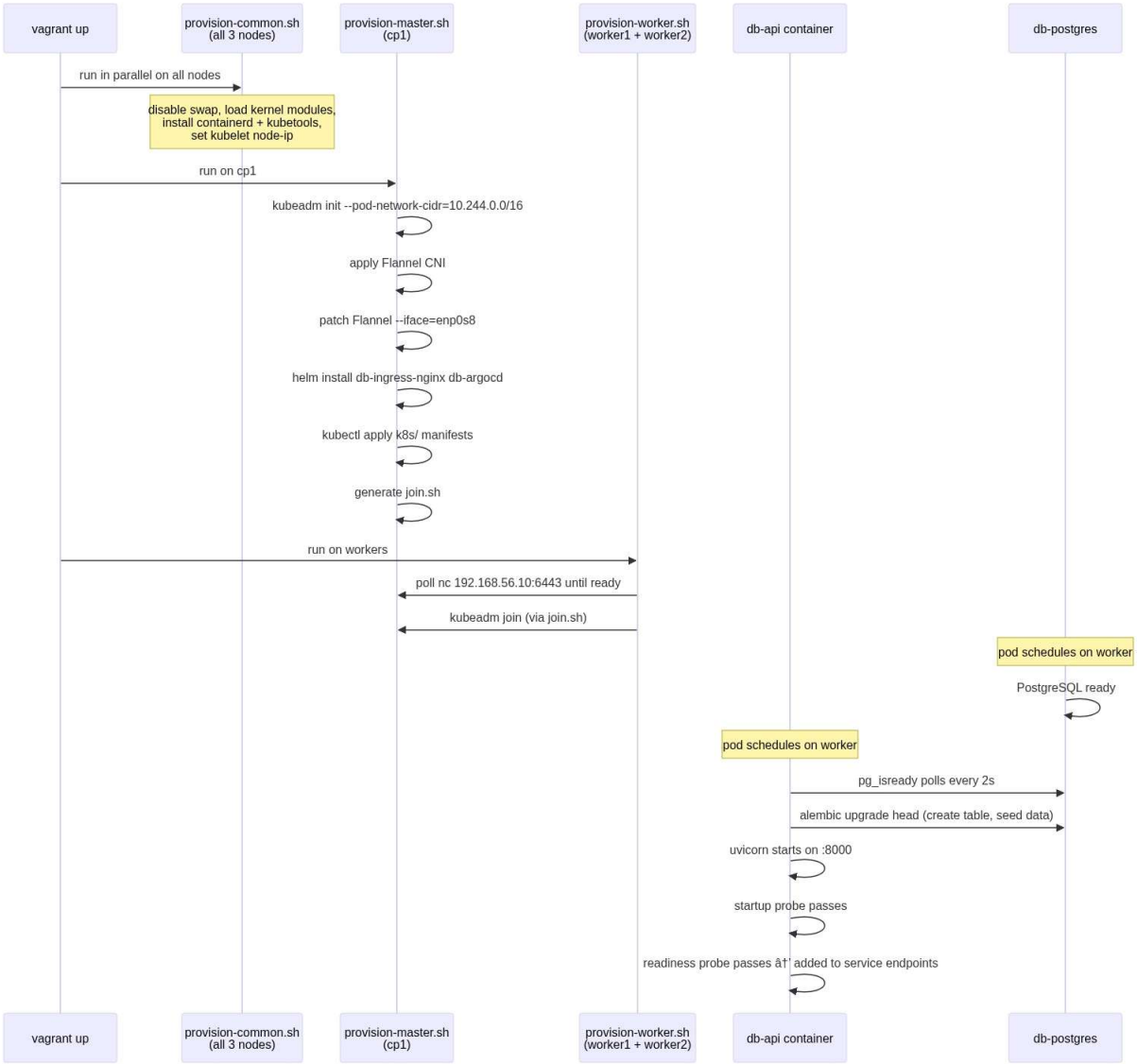
- **Application layer:** Frontend (Apache/HTML/JS) + API (FastAPI/Python) + Database (PostgreSQL)
- **Infrastructure layer:** Three-node kubeadm cluster (1 control plane + 2 workers) on VirtualBox VMs
- **Networking layer:** nginx ingress controller, Cloudflare Tunnel for public HTTPS
- **Observability layer:** Prometheus scraping four targets; Grafana with a pre-built dashboard
- **GitOps layer:** ArgoCD watching the `k8s/` directory on GitHub

1.2. Core Technologies

Layer	Technology	Version
Frontend	Apache HTTPD	2.4-alpine
API	FastAPI + Uvicorn	0.111.0 / 0.23.2
API language	Python	3.11
Database	PostgreSQL	16-alpine
ORM	SQLAlchemy	2.0.29
Migrations	Alembic	1.13.1
Metrics	prometheus-client	0.20.0
Container runtime	containerd	system

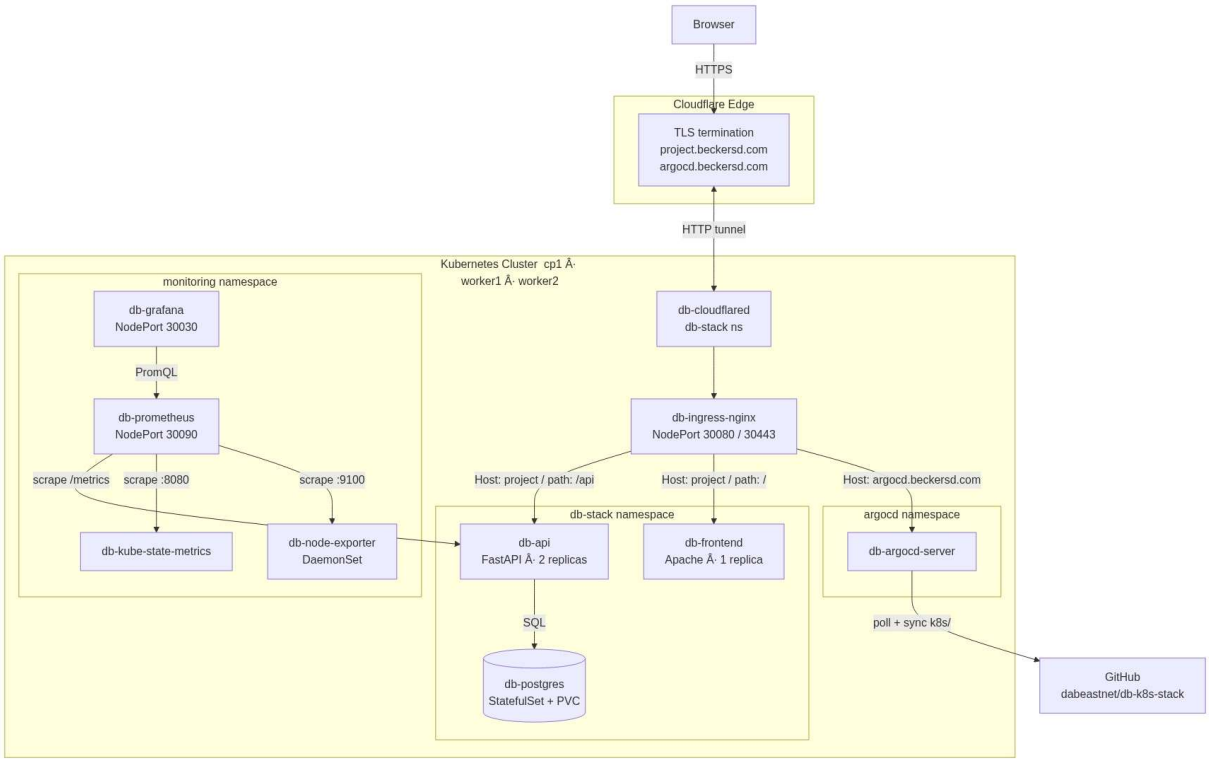
Kubernetes	kubeadm	v1.29
CNI	Flannel	latest
Ingress	ingress-nginx (Helm)	NodePort mode
GitOps	ArgoCD (Helm)	v9.x
Public TLS	Cloudflare Tunnel	cloudflared:latest
Monitoring	Prometheus	v2.51.0
Dashboards	Grafana	10.4.0
Cluster metrics	kube-state-metrics	v2.10.0
Node metrics	node-exporter	v1.8.0
Provisioning	Vagrant + VirtualBox	2.3+ / 7.0+
OS (VMs)	Ubuntu	20.04 LTS (focal64)

2. Repository Structure



3. Architecture Overview

3.1. High-Level Architecture



3.2. Kubernetes Namespaces

Namespace	Contents	Managed by
`db-stack`	frontend, API, PostgreSQL, cloudflared	deploy-k8s.sh / ArgoCD
`monitoring`	Prometheus, Grafana, kube-state-metrics, node-exporter	deploy-k8s.sh
`ingress-nginx`	nginx ingress controller	Helm (db-ingress-nginx)
`argocd`	ArgoCD components	Helm (db-argocd)

`kube-flannel`	Flannel CNI DaemonSet	kubectl apply
`kube-system`	Core Kubernetes components	kubeadm

3.3. Cluster Topology

VM	Role	IP	RAM	CPUs
cp1	control-plane	192.168.56.10	4096 MB	2
worker1	worker	192.168.56.20	4096 MB	2
worker2	worker	192.168.56.21	4096 MB	2

All VMs run Ubuntu 20.04 LTS and share the repository directory at `/vagrant`.

3.4. Port Map

Host port	VM port	Protocol	Service
18080	30080 (cp1)	HTTP	nginx ingress → frontend + API
18443	30443 (cp1)	HTTPS	nginx ingress (TLS)
19090	30090 (cp1)	HTTP	Prometheus web UI
13000	30030 (cp1)	HTTP	Grafana web UI

4. Component Analysis

4.1. Frontend (`frontend/`)

Responsibility: Serves the single-page HTML/JavaScript application that displays a greeting and container ID, both fetched live from the API.

Key files:

- `frontend/src/index.html` — the complete application
- `frontend/apache/httpd.conf` — Apache configuration
- `frontend/static/version.txt` — version string for the auto-reload mechanism
- `frontend/Dockerfile` — builds `ghcr.io/dabeastnet/db-frontend:v3`

Internal logic:

The page executes three async operations on load:

1. `fetchName()` — `GET /api/name` → populates `<span id="user">`
2. `fetchContainerId()` — `GET /api/container-id` → populates `<span id="containerId">`
3. `checkVersion()` — `GET /version.txt?_t=<timestamp>` → stores current version; if it changes on subsequent polls (every 15 seconds), calls `location.reload()`

The cache-busting `?_t=<timestamp>` query parameter prevents the browser from returning a stale cached copy of `version.txt`.

Apache proxy configuration:

Apache proxies all `/api` requests to the Kubernetes-internal DNS name:

```
ProxyPass "/api" "http://db-api.db-stack.svc.cluster.local:80/api"
```

This proxy target only resolves inside the Kubernetes cluster. In Docker Compose environments, the JavaScript fetch calls hit Apache, which tries to proxy to the K8s DNS name and fails. For local testing, API endpoints must be called directly on port 18000.

Security: Runs as UID 1001 (`appuser`). Port 8080 is used instead of 80 (non-root cannot bind below 1024). Directory listing is disabled (`Options -Indexes`).

Resource limits:

	CPU	Memory
Requests	50m	64Mi
Limits	200m	256Mi

Published image: `ghcr.io/dabeastnet/db-frontend:v3`

4.2. API (`api/`)

```
{"name":"Dieter Beckers"}
```

Responsibility: The sole component with direct database access. Exposes REST endpoints for the name and container ID, health/readiness probes for Kubernetes, and Prometheus metrics.

Key files:

- `api/app/main.py` — FastAPI app with all route handlers
- `api/app/db.py` — SQLAlchemy engine and session factory
- `api/app/models.py` — `Person` ORM model
- `api/docker-entrypoint.sh` — startup orchestration
- `api/migrations/versions/001_create_person_table.py` — schema migration + seed data
- `api/Dockerfile` — builds `ghcr.io/dabeastnet/db-api:v6`

API endpoints:

Method	Path	Description	Prometheus counter
GET	`/api/name`	Returns `{"name": "<value>"}` from first row of `person` table	`db_api_requests_total{endpoint="/api/name"}`
GET	`/api/container-id`	Returns container ID parsed from `/proc/self/cgroup`, plus `pod_name` and `hostname` from env	`db_api_requests_total{endpoint="/api/container-id"}`
GET	`/healthz`	Liveness probe — always returns `{"status": "ok"}`	—

GET	`/readyz`	Readiness probe — executes `SELECT 1`; returns 503 if DB unreachable	—
GET	`/metrics`	Prometheus text exposition format	—

Container ID parsing logic (`main.py` lines 88–102):

The endpoint reads `/proc/self/cgroup` and applies three regex patterns in order:

1. `cri-containerd-([a-f0-9]{64})\.scope` — containerd in systemd cgroup v2
2. `containerd-:/` — generic containerd
3. `\b([a-f0-9]{64})\b` — any 64-character hex string (Docker fallback)

Returns the first 12 characters of the matched ID.

Startup flow (`docker-entrypoint.sh`):

1. Apply defaults for all `DB_*` variables
2. Export `PGPASSWORD=$DB_PASSWORD` so `pg_isready` can authenticate
3. Poll `pg_isready -h $DB_HOST -p $DB_PORT -U $DB_USER` every 2 seconds until PostgreSQL accepts connections
4. Construct `DATABASE_URL` from individual `DB_*` variables if not already set
5. Run `alembic upgrade head` — idempotent migration application
6. `exec "$@"` — hands off to Uvicorn: `uvicorn app.main:app --host 0.0.0.0 --port 8000`

Database layer (`db.py`):

SQLAlchemy engine is created once at module import time with connection pooling:

- `pool_pre_ping=True` — tests connections before use, preventing stale connection errors after PostgreSQL restarts
- `pool_size=5`, `max_overflow=10` — configurable via environment variables

Sessions are provided to route handlers via FastAPI's dependency injection (`Depends(get_db)`), ensuring each request gets its own session that is reliably closed after the response.

Kubernetes probes:

Probe	Endpoint	Timing	Behaviour on failure
Startup	`/healthz`	period=10s, failureThreshold=30	Pod killed after 300 s
Readiness	`/readyz`	initialDelay=20s, period=10s	Pod removed from service endpoints
Liveness	`/healthz`	initialDelay=60s, period=20s	Pod restarted

The startup probe gives the API up to 300 seconds to initialise (covering the `pg_isready` wait and Alembic migration). Only after the startup probe passes do readiness and liveness probes begin evaluation.

Topology spread:

```
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: db-api
```

This guarantees exactly one API replica per worker node. `DoNotSchedule` prevents a third replica from being placed on a node that already has one — intentional, since the cluster has exactly two workers.

Security: Runs as UID 1001 (`appuser`). No root capabilities. Minimal base image (`python:3.11-slim`).

Resource limits:

	CPU	Memory
Requests	100m	128Mi
Limits	500m	512Mi

Published image: `ghcr.io/dabeastnet/db-api:v6`

4.3. Database (`k8s/postgres/postgres.yaml`)

Responsibility: Persistent storage of the `person` table. Managed as a Kubernetes StatefulSet with a hostPath PersistentVolume.

Resources (all in one file):

PersistentVolume `db-postgres-pv`**:**

- `capacity: 1Gi`, `accessModes: ReadWriteOnce`
- `hostPath: /mnt/postgres-data` — created on every node by `provision-common.sh`
- `storageClassName: manual` — custom label used to bind to its PVC
- `persistentVolumeReclaimPolicy: Retain` — data is not deleted when the PVC is removed

PersistentVolumeClaim `db-postgres-pvc`**:**

- Binds to `db-postgres-pv` via `storageClassName: manual`

StatefulSet `db-postgres`**:**

- Image: `postgres:16-alpine`
- Port: 5432
- `fsGroup: 999` (PostgreSQL default GID) allows the container to write to the volume
- `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` from ConfigMap/Secret
- Volume mount: `/var/lib/postgresql/data` ← PVC

Service `db-postgres`:

- ClusterIP, port 5432 → container 5432
- DNS name inside cluster: `db-postgres.db-stack.svc.cluster.local`
- API uses `DB_HOST=db-postgres` which resolves within the `db-stack` namespace

Schema:

The `person` table is created by Alembic migration `001_create_person_table`:

```
person
  id      INTEGER PRIMARY KEY AUTOINCREMENT
  name    VARCHAR(255) NOT NULL
```

One row is seeded on first run: `id=1, name='Dieter Beckers'`. All API reads use `SELECT * FROM person LIMIT 1`. Name updates target `WHERE id = 1`.

4.4. nginx Ingress (`k8s/ingress/ingress.yaml`)

Responsibility: Routes incoming HTTP requests to the correct Kubernetes service based on `Host` header and URL path.

Ingress class: `nginx` (served by the `db-ingress-nginx` Helm release in the `ingress-nginx` namespace)

Routing rules:

Rule 1 — named host `project.beckersd.com`:

Path	PathType	Backend
`/api`	Prefix	`db-api:80`
`/`	Prefix	`db-frontend:80`

Rule 2 — catch-all (no host), used for `localhost:18080`:

Path	PathType	Backend
`/api`	Prefix	`db-api:80`
`/`	Prefix	`db-frontend:80`

The catch-all rule matches any `Host` header, making `http://localhost:18080` work without configuring a custom Host header.

Annotations:

- `ssl-redirect: "false"` — disabled while TLS is handled by Cloudflare
- `proxy-body-size: 4m` — maximum request body size

TLS (commented out): The TLS block referencing `cert-manager.io/cluster-issuer: letsencrypt-prod` is commented out. Enabling it requires installing cert-manager and providing a real email in `k8s/cert-manager/clusterissuer.yaml`.

4.5. Cloudflare Tunnel (`k8s/cloudflared/deployment.yaml`)

Responsibility: Provides public HTTPS access to the cluster without requiring a public IP, open firewall ports, or a static IP address. Cloudflare terminates TLS at the edge.

How it works:

1. The `db-cloudflared` pod maintains a persistent outbound connection to Cloudflare's global network using the provided tunnel token
2. When a browser requests `https://project.beckersd.com`, Cloudflare routes the request through this tunnel to the pod
3. The pod forwards the request to `http://db-ingress-nginx-controller.ingress-nginx.svc.cluster.local` (configured in the Cloudflare dashboard, not in these manifests)
4. The nginx ingress routes to `db-frontend` or `db-api` based on the path

Resources:

- **Secret** `db-cloudflared-token`: stores the Cloudflare Tunnel token in `stringData.token`
- **Deployment** `db-cloudflared`: runs `cloudflare/cloudflared:latest` with `tunnel --no-autoupdate run`; the token is injected as the `TUNNEL_TOKEN` environment variable from the Secret

To update the tunnel token: edit line 9 of `k8s/cloudflared/deployment.yaml`, then run:

```
kubectl apply -f k8s/cloudflared/deployment.yaml
kubectl rollout restart deployment db-cloudflared -n db-stack
```

4.6. Monitoring Stack (`k8s/monitoring/`)

Responsibility: Full observability of cluster resources and application performance without requiring the Prometheus Operator.

Components:

Prometheus (`prometheus.yaml`):

Static scrape configuration targeting four jobs:

Job	Target	What it collects
`prometheus`	`localhost:9090`	Self-metrics
`kube-state-metrics`	`db-kube-state-metrics.monitoring:8080`	K8s object state
`db-api`	`db-api.db-stack:80/metrics`	Application request counters
`kubernetes-nodes`	All nodes via K8s SD on port 9100	node-exporter hardware metrics

The `kubernetes-nodes` job uses Kubernetes service discovery to dynamically discover all nodes and relabels `__address__` to `<node-IP>:9100`. Node names (`cp1`, `worker1`, `worker2`) are preserved as the `node` label.

Service: NodePort 30090 → `http://localhost:19090`

kube-state-metrics (`kube-state-metrics.yaml`):

Exposes Kubernetes object state as Prometheus metrics. Key metrics: `kube_pod_status_phase`, `kube_node_status_condition`, `kube_pod_container_status_restarts_total`.

node-exporter (`node-exporter.yaml`):

DaemonSet running one pod per node (including cp1 via a control-plane toleration). Uses `hostNetwork: true`, `hostPID: true`, and read-only mounts of `/proc`, `/sys`, `/` to read kernel-level hardware counters.

Grafana (`grafana.yaml`):

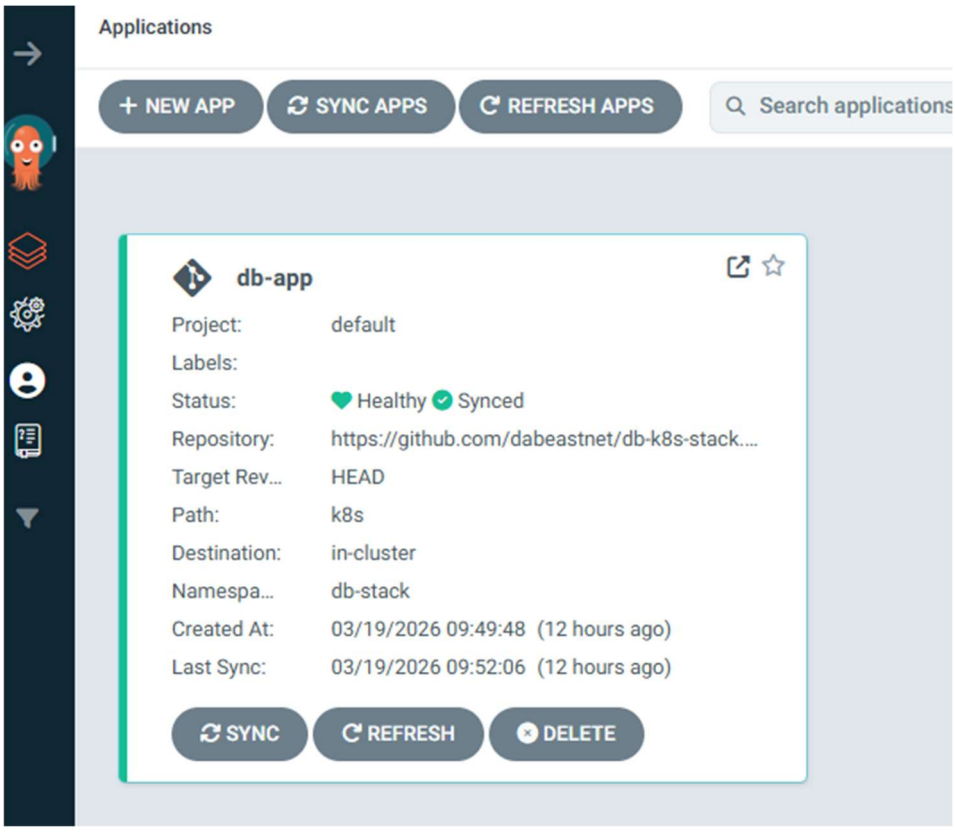
Pre-provisioned via three ConfigMaps:

ConfigMap	Purpose
`db-grafana-datasources`	Auto-registers Prometheus as default datasource
`db-grafana-dashboard-provider`	Configures file-based dashboard loading
`db-grafana-dashboard`	Pre-built "db-k8s-stack Overview" dashboard JSON

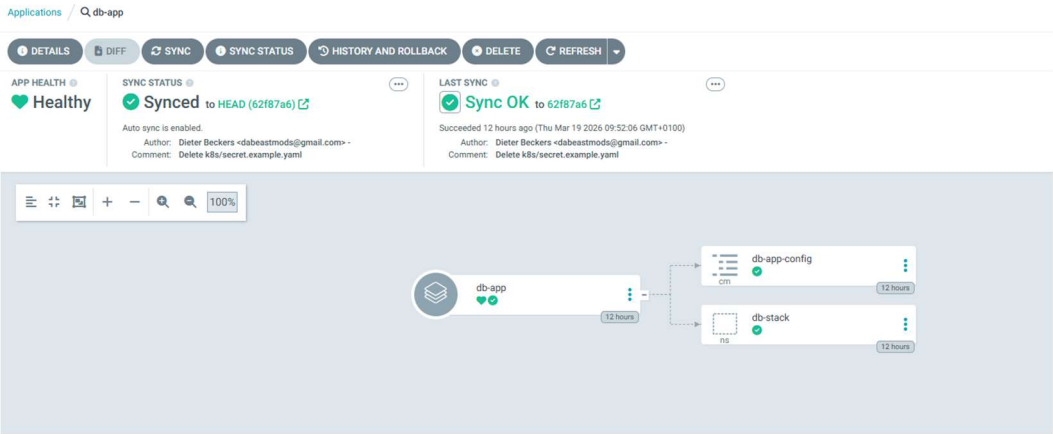
Dashboard panels: Running Pods, Ready Nodes, API Request Rate, Pod Restarts, Node CPU %, Node Memory %, API Request Rate per endpoint, Node Disk I/O.

Service: NodePort 30030 → `http://localhost:13000` (admin / admin)

4.7. ArgoCD (`helm/argocd-values.yaml`, `k8s/argocd/application.yaml`)



Responsibility: GitOps controller — continuously reconciles the cluster state with the `k8s/` directory of the GitHub repository.



Helm release: `db-argocd` in namespace `argocd`

Key configuration decisions:

- `--insecure` mode: ArgoCD serves plain HTTP; TLS is terminated by Cloudflare Tunnel upstream

- `global.tolerations`: allows pre-install Jobs to schedule on cp1 before workers join (otherwise Helm install times out)
- `server.ingress.hostname: argocd.beckersd.com`: direct subdomain required because Cloudflare Universal SSL wildcard `*.beckersd.com` does not cover two levels deep (e.g., `argocd.project.beckersd.com` would not be covered)

ArgoCD Application (`db-app`):

```
source:
  repoURL: https://github.com/dabeastnet/db-k8s-stack.git
  targetRevision: HEAD
  path: k8s
syncPolicy:
  automated:
    prune: true      # resources deleted from Git are deleted from cluster
    selfHeal: true   # manual cluster changes are reverted to match Git
```

ArgoCD polls every ~3 minutes. Any commit to `k8s/` on the `main` branch is automatically applied to the cluster.

Retrieving the admin password:

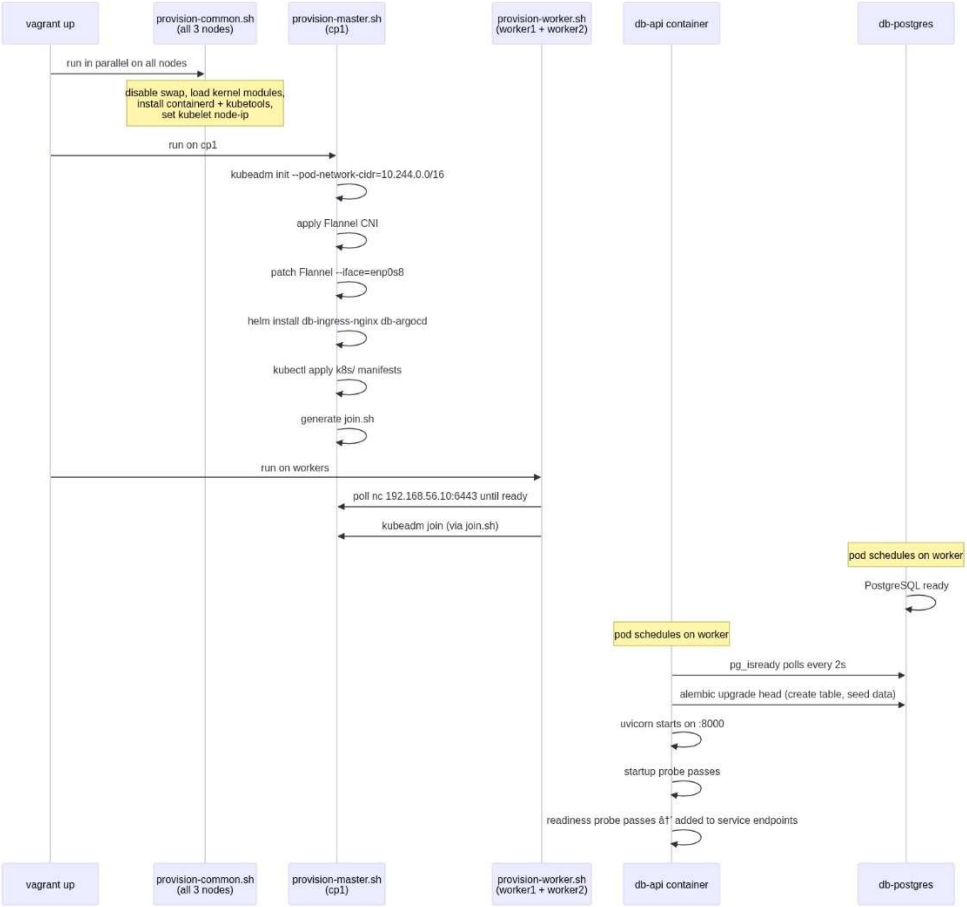
```
vagrant ssh cp1
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d && echo
```

The password changes on every cluster rebuild (new secret generated by Helm).

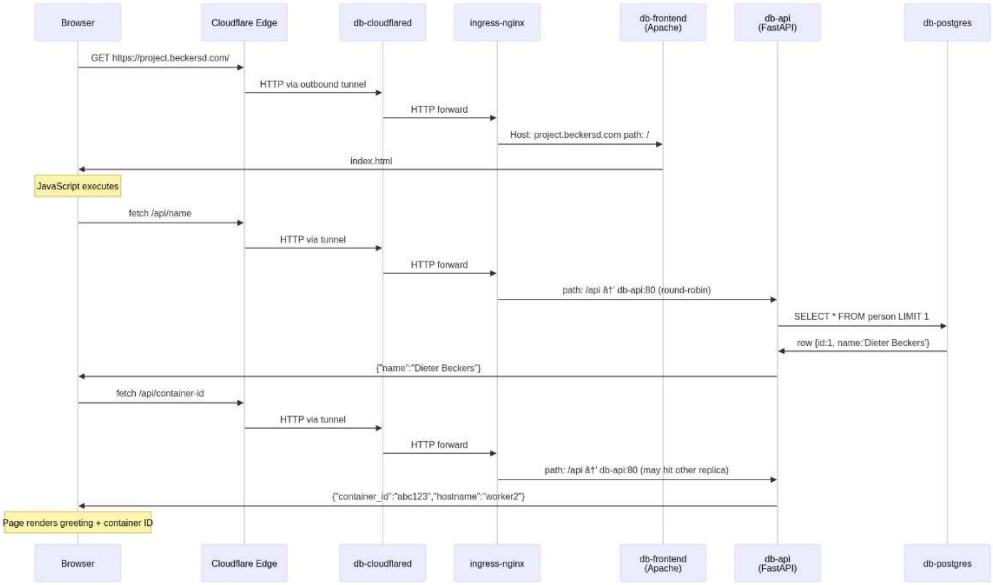
---

5. Runtime and Request Flows

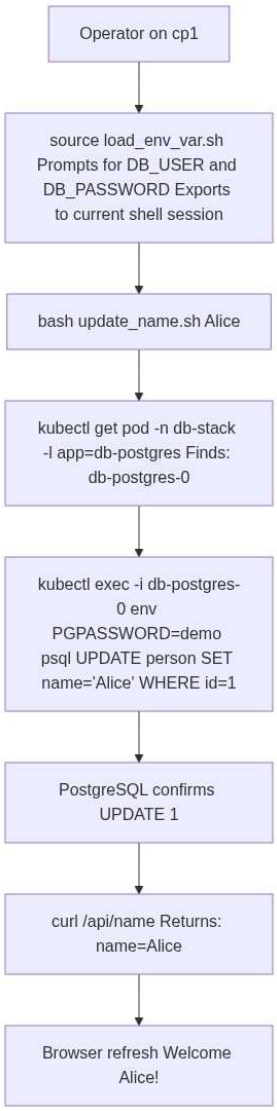
5.1. Application Start Sequence (fresh `vagrant up`)



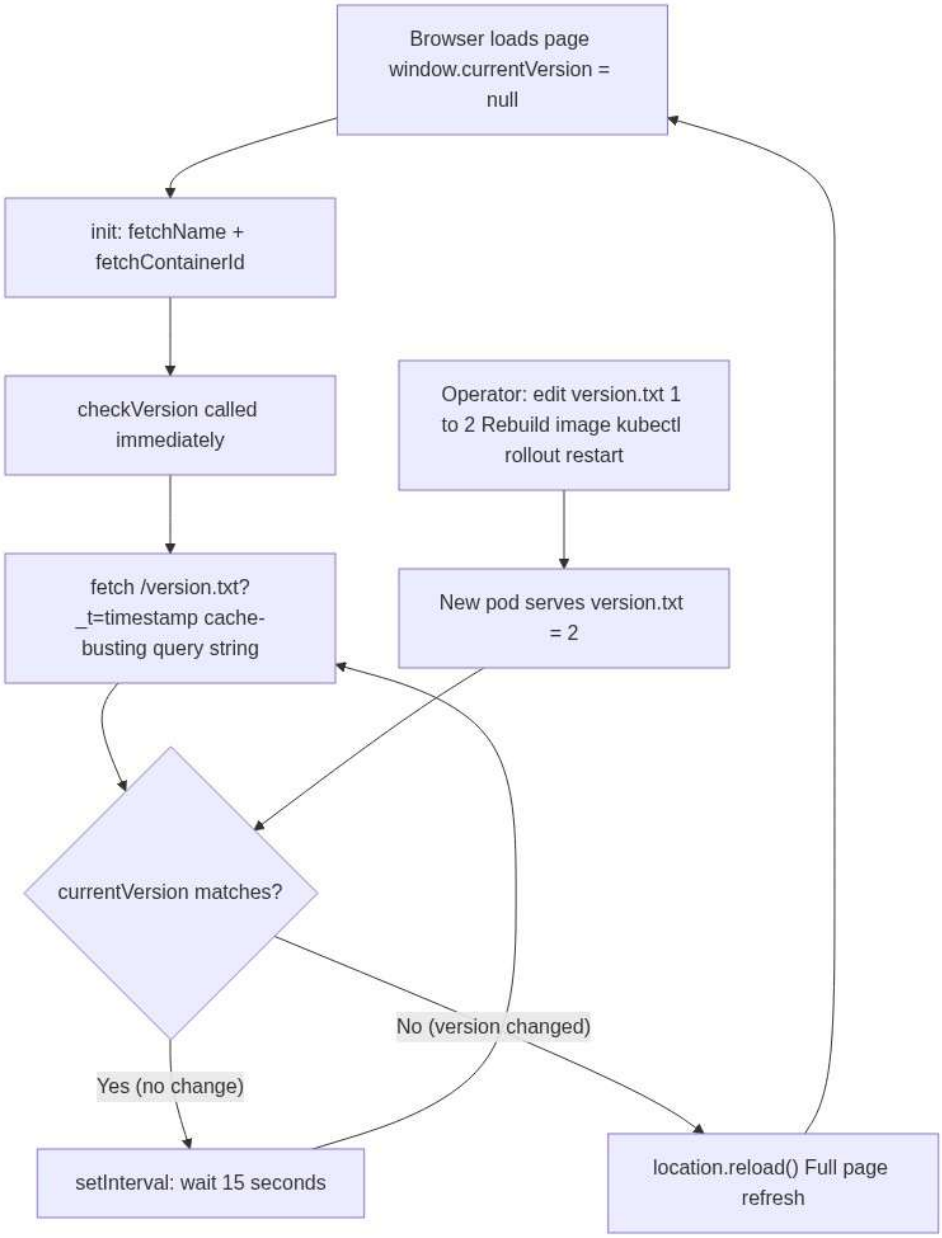
5.2. HTTP Request Flow (browser → database)



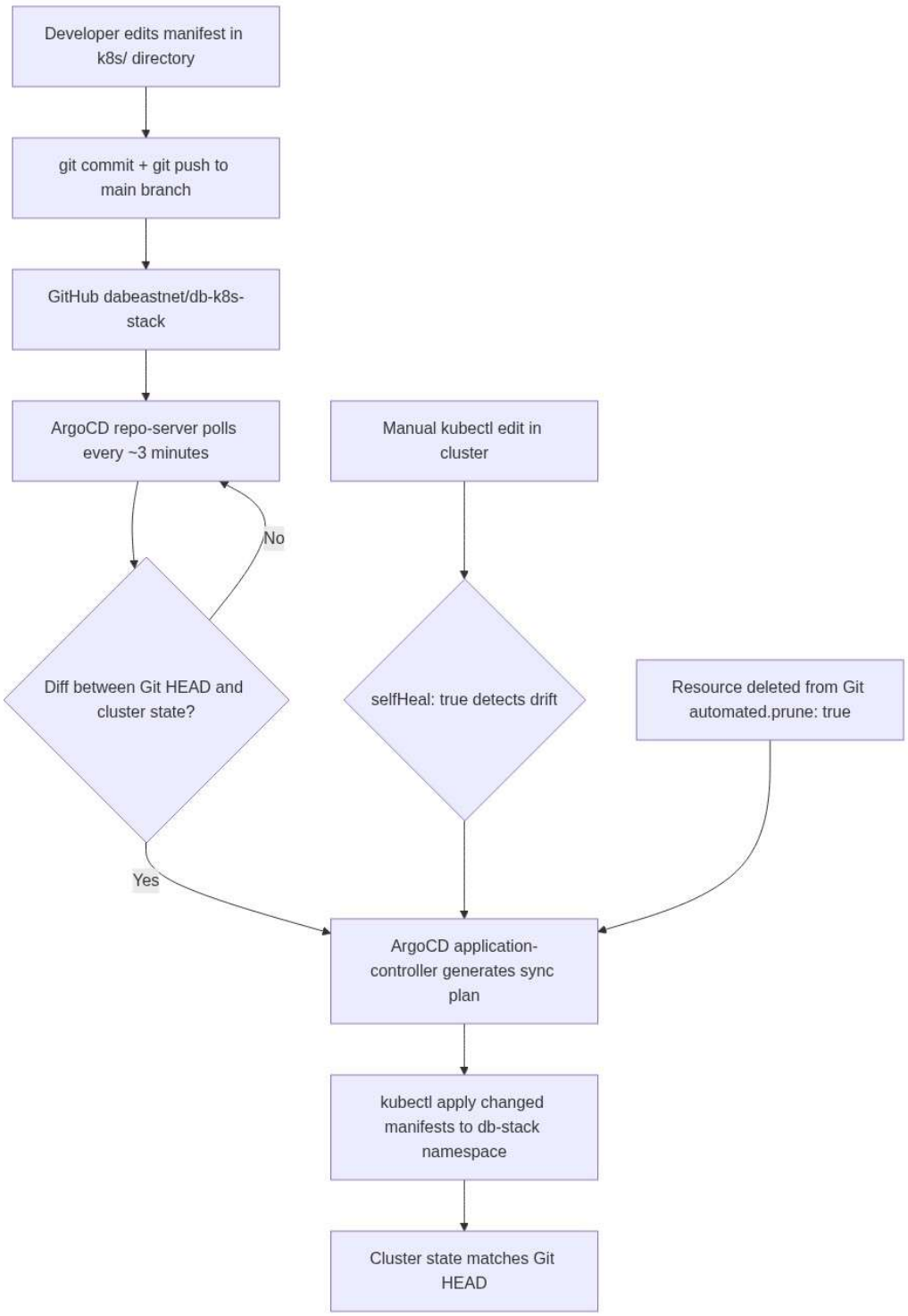
5.3. Name Update Flow



5.4. Auto Layout Refresh Flow



5.5. GitOps Sync Flow



6. Configuration and Environment Variables

6.1. Application Environment Variables

Variable	Default	Source in K8s	Description
`DB_HOST`	`db-postgres`	ConfigMap `db-app-config`	PostgreSQL service hostname
`DB_PORT`	`5432`	ConfigMap `db-app-config`	PostgreSQL port
`DB_NAME`	`demo`	ConfigMap `db-app-config`	Database name
`DB_USER`	`demo`	Secret `db-app-secret`	Database username
`DB_PASSWORD`	`demo`	Secret `db-app-secret`	Database password
`PGPASSWORD`	(= DB_PASSWORD)	Secret `db-app-secret`	Used by `pg_isready` in entrypoint
`DATABASE_URL`	constructed	—	Full SQLAlchemy URL; auto-built if absent
`DB_POOL_SIZE`	`5`	—	SQLAlchemy connection pool size
`DB_MAX_OVERFLOW`	`10`	—	SQLAlchemy max extra connections
`POD_NAME`	`""`	Downward API (`metadata.name`)	Pod name; returned by `/api/container-id`
`HOSTNAME`	`""`	Downward API (`spec.nodeName`)	Node name; returned by `/api/container-id`

6.2. Kubernetes ConfigMap (`k8s/configmap.yaml`)

`db-app-config` holds non-sensitive database connection parameters:

Key	Value
`DB_HOST`	`db-postgres`
`DB_PORT`	`5432`
`DB_NAME`	`demo`

6.3. Kubernetes Secret (`k8s/secret.example.yaml`)

`db-app-secret` holds database credentials. The example file contains development values only:

Key	Example value
`DB_USER`	`demo`
`DB_PASSWORD`	`demo`

Important: This file is a template. In `provision-master.sh`, the secret is only applied if `db-app-secret` does not already exist — allowing real credentials to be pre-created before provisioning.

6.4. `.env.example`

Template for local development. Variables:

```
DB_HOST=db-postgres
DB_PORT=5432
DB_NAME=demo
DB_USER=demo
DB_PASSWORD=demo
DOMAIN=example.com
EMAIL=admin@example.com
GIT_REPO_URL=https://github.com/your-user/db-k8s-stack.git
GIT_REPO_BRANCH=main
```

6.5. Configuration Risks and Gotchas

- **Default credentials in Git:** `secret.example.yaml` contains `demo/demo`. The provisioning script guards against overwriting an existing secret, but the default credentials are not suitable for production.
- **Cloudflare token in Git:** `k8s/cloudflared/deployment.yaml` currently contains the Cloudflare Tunnel token directly in the manifest. This should be managed outside of Git (e.g., via a pre-created Secret) in a production environment.
- **cert-manager email placeholder:** `k8s/cert-manager/clusterissuer.yaml` contains `admin@example.com` — must be replaced before installing cert-manager.
- **ArgoCD initial password:** The auto-generated password in `argocd-initial-admin-secret` should be changed after first login.
- **emptyDir storage for Prometheus and Grafana:** All monitoring data is lost on pod restart. Replace with PVCs for persistence in a production setup.

---

7. API and Interfaces

7.1. REST API Reference

Base URL (local): `http://localhost:18080` (via ingress) or `http://localhost:18000` (direct, Docker Compose only)

GET /api/name

Returns the name stored in the database.

Response:

```
{ "name": "Dieter Beckers" }
```

If the `person` table is empty: `{ "name": "Unknown" }`.

GET /api/container-id

Returns the running container's identity.

Response:

```
{
  "container_id": "abc123def456",
  "pod_name": "db-api-6d9f7b5c4-xk9pq",
  "hostname": "worker1"
}
```

`container_id` is derived from `/proc/self/cgroup` (12 characters). `pod_name` and `hostname` are injected via the Kubernetes Downward API.

GET /healthz

Liveness probe. Always returns 200.

```
{ "status": "ok" }
```

GET /readyz

Readiness probe. Executes `SELECT 1` against PostgreSQL.

- 200: `{ "status": "ok" }` — database reachable
- 503: `{ "detail": "Database not ready" }` — database unreachable

GET /metrics

Prometheus text exposition format. Key metric:

```
# HELP db_api_requests_total Total API requests
# TYPE db_api_requests_total counter
db_api_requests_total{endpoint="/api/name",method="GET"} 4.0
db_api_requests_total{endpoint="/api/container-id",method="GET"} 2.
0
```

7.2. Inter-Service Communication

Source	Target	Protocol	Address
Browser JS	`/api/*`	HTTP (via ingress)	Relative URL — same origin
Apache	db-api	HTTP	`db-api.db-stack.svc.cluster.local:80`
db-api	db-postgres	PostgreSQL	`db-postgres.db-stack.svc.cluster.local:5432`
Prometheus	db-api	HTTP	`db-api.db-stack.svc.cluster.local:80/metrics`
Prometheus	kube-state-metrics	HTTP	`db-kube-state-metrics.monitoring.svc.cluster.local:8080`
Prometheus	node-exporter	HTTP	Node IP:9100 (Kubernetes SD)
Grafana	Prometheus	HTTP	`db-prometheus.monitoring.svc.cluster.local:9090`
ArgoCD	GitHub	HTTPS	`github.com/dabeastnet/db-k8s-stack.git`
cloudflared	ingress-nginx	HTTP	`db-ingress-nginx-controller.ingress-nginx.svc.cluster.local`

7.3. Authentication and Security

- No application-level authentication on the frontend or API (read-only public application)
- Database credentials are stored in a Kubernetes Secret (base64-encoded by Kubernetes)
- Cloudflare Tunnel token stored in a Kubernetes Secret
- ArgoCD protected by initial admin password
- Grafana protected by `admin/admin` (should be changed)

- All pods run as non-root (UID 1001 for frontend and API; UID 999 for PostgreSQL; UID 472 for Grafana)
- Minimal base images reduce attack surface

---

8. Database and Persistence

8.1. PostgreSQL

- **Image:** `postgres:16-alpine`
- **Deployment model:** Kubernetes StatefulSet (`db-postgres`) with 1 replica
- **Storage:** hostPath PV at `/mnt/postgres-data` on the node where the pod schedules
- **Service:** ClusterIP `db-postgres`, port 5432

8.2. Schema

```
CREATE TABLE person (
    id   SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);
INSERT INTO person (name) VALUES ('Dieter Beckers');
```

Created and seeded by Alembic migration `001_create_person_table`. Alembic tracks applied migrations in the `alembic_version` table. Running `alembic upgrade head` on an already-migrated database is safe and idempotent.

8.3. Read/Write Patterns

Operation	Query	Trigger
Read name	`SELECT * FROM person LIMIT 1`	Every `GET /api/name` request
Write name	`UPDATE person SET name = :name WHERE id = 1`	`update_name.sh` via `kubectl exec`

Health check	`SELECT 1`	Every `GET /readyz` request

8.4. Persistence Model

The hostPath PV (`/mnt/postgres-data`) persists across pod restarts because the directory lives on the node's filesystem. However, if the PostgreSQL pod is rescheduled to a different node, the data will not follow it. For a production setup, a distributed storage solution (e.g., Rook-Ceph, Longhorn) should be used instead.

The `Retain` reclaim policy ensures data is not deleted when the PVC is removed — manual cleanup of the directory is required if the PV is to be reused.

---

9. Build, Run, Test, and Deploy

9.1. Building Images

```
./build.sh
```

Builds:

- `db-frontend:latest` from `./frontend`
- `db-api:latest` from `./api`

To push to a registry:

```
export REGISTRY=ghcr.io/dabeastnet
./push.sh
```

9.2. Local Development (Docker Compose)

```
docker compose up --build
```

Service	Host port	Notes
db-api	18000	Direct API access for testing
db-frontend	18080	Apache frontend
db-postgres	—	Internal only

Limitation: The Apache proxy (`/api` → `db-api.db-stack.svc.cluster.local`) does not resolve outside Kubernetes. Test API endpoints directly on port 18000.

Full workflow: see `LOCAL_TESTING.md`.

9.3. Running the Full Smoke Test

Docker Compose:

```
./test-local.sh
```

Kubernetes:

```
./test-k8s.sh
```

Smoke tests hit all endpoints and print responses. No assertions — review output manually.

9.4. Deploying to Kubernetes

Full cluster provisioning (from scratch):

```
vagrant up
```

Apply manifests to a running cluster:

```
./deploy-k8s.sh
```

Apply individual components:

```
vagrant ssh cp1 -- kubectl apply -f /vagrant/k8s/api/deployment.yam
l
```

9.5. Cluster Lifecycle Commands

```
vagrant up              # Create and provision all VMs
vagrant halt            # Stop all VMs
vagrant destroy -f      # Delete all VMs
vagrant provision cp1   # Re-run provisioner on cp1
vagrant ssh cp1         # SSH into cp1
vagrant status          # Check VM states
```

9.6. Updating the Application Name

```
vagrant ssh cp1
source /vagrant/vagrant/scripts/load_env_var.sh
# Enter DB_USER: demo
# Enter DB_PASSWORD: demo
bash /vagrant/vagrant/scripts/update_name.sh "Alice"
```

---

10. Kubernetes and Infrastructure

10.1. Kubernetes Version and Installation

Kubernetes v1.29 is installed via the official `pkgs.k8s.io` APT repository. The package pinning for `kubelet`, `kubeadm`, and `kubectl` uses `apt-mark hold` (commented out in the provisioner — a minor observation).

10.2. Network Architecture

All Vagrant VMs have two NICs:

Interface	Type	Address	Purpose
`enp0s3`	VirtualBox NAT	10.0.2.x (same for all VMs)	Internet access
`enp0s8`	Host-only	192.168.56.x (unique per VM)	VM-to-VM communication

Critical: Flannel auto-detects `enp0s3` (the first interface), which cannot route traffic between VMs. The provisioner patches the Flannel DaemonSet with `--iface=enp0s8` to force the correct interface. Without this, all cross-node pod communication (including DNS) fails silently.

Kubelet is configured with `--node-ip=192.168.56.x` (written to `/etc/default/kubelet`) to ensure each node registers with its unique private IP rather than the shared NAT address.

10.3. CNI — Flannel

- Applied from: `https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml`
- Patched post-install: `--iface=enp0s8` appended to DaemonSet args
- Pod CIDR: `10.244.0.0/16` (set at `kubeadm init`)

- Encapsulation: VXLAN over the private network

10.4. Helm Releases

Release	Chart	Namespace	Key settings
`db-ingress-nginx`	`ingress-nginx/ingress-nginx`	`ingress-nginx`	NodePort 30080/30443, webhooks disabled
`db-argocd`	`argo/argo-cd`	`argocd`	`--insecure`, global tolerations, ingress hostname

10.5. Resource Limits Summary

Pod	CPU request	CPU limit	Memory request	Memory limit
db-api	100m	500m	128Mi	512Mi
db-frontend	50m	200m	64Mi	256Mi
db-cloudflared	50m	200m	64Mi	128Mi

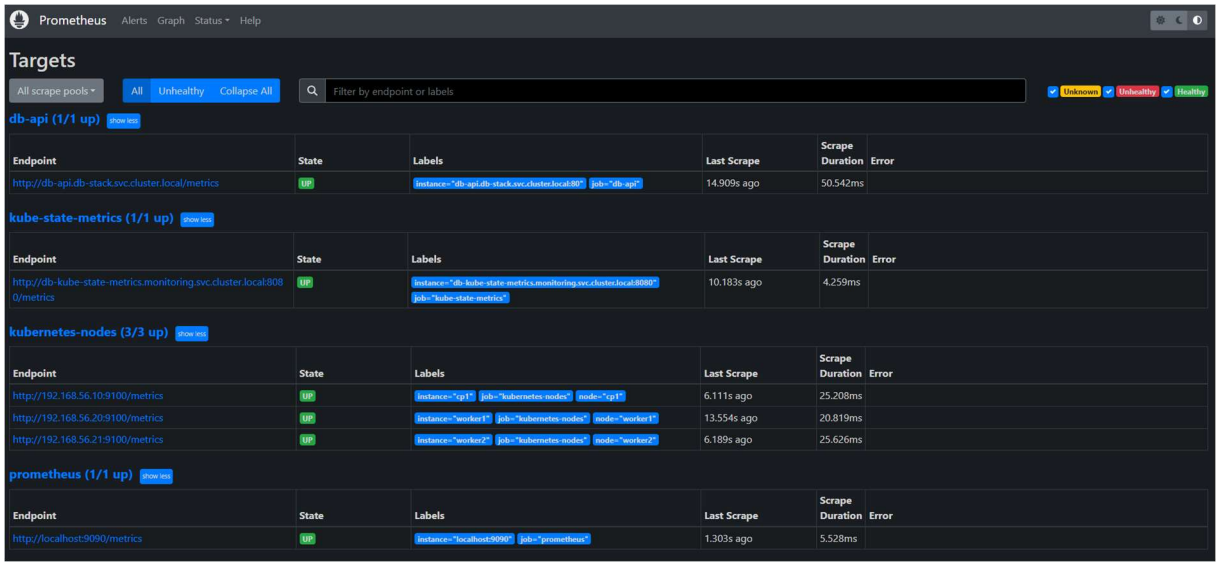
11. Monitoring, Logging, and Operations

11.1. Prometheus

Access: `http://localhost:19090`

Key queries:

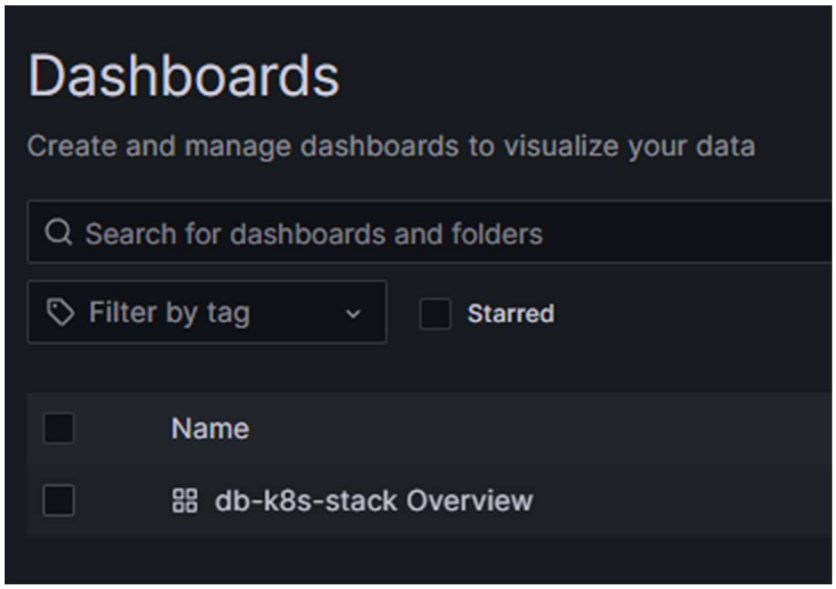
Query	Description
`db_api_requests_total`	All API request counts by endpoint
`rate(db_api_requests_total[5m])`	Request rate per second
`sum(kube_pod_status_phase{namespace="db-stack",phase="Running"})`	Running pods
`count(kube_node_status_condition{condition="Ready",status="true"})`	Ready nodes
`100 * (1 - avg by(node)(rate(node_cpu_seconds_total{mode="idle"}[5m])))`	Node CPU %



11.2. Grafana

Access: `http://localhost:13000` (admin / admin)

The pre-built dashboard "db-k8s-stack Overview" shows eight panels covering pod health, node resources, API performance, and disk I/O.



11.3. Logging

All containers log to stdout/stderr (container runtime captures these):

```
# API logs
kubectl logs -n db-stack -l app=db-api --tail=50

# Frontend logs
kubectl logs -n db-stack -l app=db-frontend --tail=50

# PostgreSQL logs
kubectl logs -n db-stack db-postgres-0 --tail=50

# Cloudflare tunnel
kubectl logs -n db-stack -l app=db-cloudflared --tail=20

# Prometheus
kubectl logs -n monitoring -l app=db-prometheus --tail=50
```

API logging format: `%(asctime)s %(levelname)s [%(name)s] %(message)s` at INFO level. Warnings for container ID parse failures; errors for readiness check failures.

11.4. Health Checks and Restarts

- **Startup probe** prevents premature liveness evaluation during the initial `pg_isready` wait and Alembic migration (up to 300 s grace)
- **Readiness probe** removes the API from the service's endpoint list when the database is unreachable — traffic automatically redirects to the healthy replica
- **Liveness probe** triggers a pod restart if the API becomes unresponsive; the 60-second `initialDelaySeconds` prevents premature restarts during startup

11.5. Common Failure Points

Symptom	Likely cause	Fix
API pods `Pending`	worker2 not joined	`vagrant up worker2`
`bridge fdb show` shows `10.0.2.x`	Flannel on wrong NIC	Re-provision cp1
ArgoCD redirects to main site	Incorrect ingress hostname	Check ArgoCD ingress host
Cloudflare `tunnel error`	Invalid token or wrong origin URL	Update token; check dashboard origin URL
Workers fail to join	API server not ready when workers boot	`provision-worker.sh` poll loop handles this

12. Key Scripts and Automation

12.1. `vagrant/provision-common.sh`

Runs on every node. Responsibilities:

1. Disable swap (`swapoff -a`, comment out in `/etc/fstab`)
2. Install APT prerequisites
3. Load `overlay` and `br_netfilter` kernel modules permanently
4. Set sysctl: `bridge-nf-call-iptables`, `ip_forward`
5. Install and configure containerd (`SystemdCgroup = true`)
6. Create `/mnt/postgres-data`
7. Add Kubernetes v1.29 APT repository
8. Install `kubelet`, `kubeadm`, `kubectl`, `cri-tools`, `conntrack`, `kubernetes-cni`
9. Set `KUBELET_EXTRA_ARGS=--node-ip=192.168.56.x`

12.2. `vagrant/provision-master.sh`

Runs on cp1 only. Responsibilities:

1. `kubeadm init` (only if `/etc/kubernetes/admin.conf` does not exist)
2. Set up kubeconfig for the `vagrant` user
3. Apply Flannel; patch DaemonSet with `--iface=enp0s8`; wait for rollout
4. Install Helm
5. Install `db-ingress-nginx` and `db-argocd` Helm releases
6. Wait for `applications.argoproj.io` CRD to be registered
7. Generate `join.sh`
8. Apply all core K8s manifests (namespace through ingress)
9. Call `bash /vagrant/deploy-k8s.sh`

12.3. `vagrant/provision-worker.sh`

Runs on worker1 and worker2. Polls `192.168.56.10:6443` with `nc` until the API server is reachable, then executes `bash join.sh --v=5`.

12.4. `deploy-k8s.sh`

Applies all manifests in dependency order. Non-fatal blocks for cert-manager ClusterIssuers, Prometheus Operator ServiceMonitor, and ArgoCD Application (skipped gracefully if CRDs are absent). Uses `SCRIPT_DIR` for absolute path resolution — works when called from any directory by Vagrant provisioners.

12.5. `vagrant/scripts/update_name.sh`

1. Validates that `$1` (new name), `DB_USER`, `DB_NAME`, `DB_PASSWORD` are set
2. Finds the `db-postgres-0` pod by label `app=db-postgres`
3. Executes `UPDATE person SET name = :name WHERE id = 1` via `kubectl exec` + `psql`

Must be preceded by `source vagrant/scripts/load_env_var.sh` to set credentials interactively.

12.6. `package.sh`

Creates `db-k8s-stack.zip` from the repository root, excluding `.git`, `*.pyc`, `__pycache__`, `vendor/`, `node_modules/`, and the archive itself.

---

13. Risks, Observations, and Improvement Opportunities

13.1. Confirmed Issues

Item	File	Description
Cloudflare token in Git	`k8s/cloudflared/deployment.yaml:9`	Tunnel token is committed in plaintext. Should be managed via a pre-created Secret outside the repository.
Default credentials in Git	`k8s/secret.example.yaml`	`demo/demo` credentials in the repository. Acceptable for a dev cluster; not suitable for production.
cert-manager placeholder email	`k8s/cert-manager/clusterissuer.yaml:9`	`admin@example.com` must be replaced before cert-manager is installed.
Stale Vagrantfile comment	`Vagrantfile:21`	Comment references `app.example.com` (old hostname) after domain was changed to `project.beckersd.com`.
Commented-out `apt-mark hold`	`provision-common.sh:58`	Without package holds, future `apt-get upgrade` runs could update Kubernetes packages to an incompatible version.
`set -euo pipefail` commented out	`vagrant/scripts/update_name.sh:5`	Weakened error handling; errors in sub-commands may be silently ignored.

13.2. Observations / Inferred Risks

Item	Description	Risk
`imagePullPolicy: IfNotPresent`	API deployment uses cached images. If the image tag is not updated after a rebuild, the new code will not be deployed.	Medium
emptyDir for Prometheus and Grafana	All metrics and user-created dashboards are lost on pod restart.	Low in dev, high in production.
Single PostgreSQL replica	No replication or standby. Pod failure = downtime until Kubernetes reschedules.	Acceptable for dev.
hostPath PV not portable	If PostgreSQL reschedules to a different node, data is lost.	Medium
ArgoCD `admin@example.com`	The ArgoCD Application manifest has a placeholder repo URL comment. The actual URL is correct, but the comment may mislead.	Low
No rate limiting on ingress	`/api` endpoints have no rate limiting configured on the ingress.	Low for this use case.
Docker Compose port mismatch in old test-local.sh defaults	Fixed (18000/18080). Original defaults were 8000/8080.	Fixed

13.3. Improvement Opportunities

- Replace hostPath PV with a distributed storage solution for PostgreSQL portability
- Move the Cloudflare tunnel token to a pre-provisioned Secret not stored in Git
- Add `apt-mark hold kubelet kubeadm kubectl` to lock package versions
- Enable cert-manager for in-cluster Let's Encrypt certificate management
- Add Grafana persistent storage to retain user-created dashboards

- Consider Horizontal Pod Autoscaler for the API deployment
- Add network policies to restrict pod-to-pod communication to required paths only

---

14. Appendix A: Important Files

File	Purpose
`Vagrantfile`	Cluster topology and port forwarding
`vagrant/provision-common.sh`	Node baseline: swap, kernel, containerd, kubetools
`vagrant/provision-master.sh`	Control plane: kubeadm init, CNI, Helm, manifests
`vagrant/provision-worker.sh`	Worker join with API server readiness polling
`vagrant/scripts/update_name.sh`	Update name in PostgreSQL via kubectl
`vagrant/scripts/load_env_var.sh`	Interactive credential loader for shell session
`api/app/main.py`	All FastAPI route handlers
`api/app/db.py`	SQLAlchemy engine + session factory
`api/app/models.py`	Person ORM model
`api/docker-entrypoint.sh`	DB wait + migration + server start
`api/migrations/versions/001_create_person_table.py`	Schema + seed data
`frontend/src/index.html`	Complete single-page application
`frontend/apache/httpd.conf`	Apache config with /api proxy

`frontend/static/version.txt`	Auto-reload version trigger
`k8s/namespace.yaml`	db-stack namespace
`k8s/configmap.yaml`	DB host/port/name (non-secret)
`k8s/secret.example.yaml`	DB user/password template
`k8s/api/deployment.yaml`	API: 2 replicas, topology spread, all probes
`k8s/api/service.yaml`	db-api ClusterIP
`k8s/frontend/deployment.yaml`	Frontend deployment
`k8s/frontend/service.yaml`	db-frontend ClusterIP
`k8s/postgres/postgres.yaml`	PV + PVC + StatefulSet + Service
`k8s/ingress/ingress.yaml`	nginx routing rules
`k8s/cloudflared/deployment.yaml`	Cloudflare Tunnel token + Deployment
`k8s/argocd/application.yaml`	ArgoCD GitOps Application
`k8s/cert-manager/clusterissuer.yaml`	Let's Encrypt prod + staging
`k8s/monitoring/prometheus.yaml`	Static scrape config + Deployment + NodePort
`k8s/monitoring/grafana.yaml`	Pre-provisioned dashboard
`k8s/monitoring/node-exporter.yaml`	Per-node hardware metrics DaemonSet
`k8s/monitoring/kube-state-metrics.yaml`	K8s object state metrics
`helm/argocd-values.yaml`	ArgoCD Helm values

`deploy-k8s.sh`	Apply all manifests in order
`docker-compose.yml`	Local development stack
`.env.example`	Environment variable template
`VERIFICATION.md`	Per-criterion verification guide
`LOCAL_TESTING.md`	Docker Compose testing guide

---

15. Appendix B: Commands Reference

15.1. Cluster Management

```
vagrant up                          # Create and provision full clu
ster
vagrant halt                        # Stop all VMs
vagrant destroy -f                  # Delete all VMs and free disk
vagrant up worker2                  # Start/provision a single VM
vagrant provision cp1               # Re-run provisioner on cp1
vagrant ssh cp1                     # SSH into control plane
vagrant ssh cp1 -- kubectl get nodes # Run kubectl without interact
ive SSH
vagrant status                      # Show VM states
```

15.2. Kubernetes — Cluster Inspection

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get pods -n db-stack -o wide
kubectl get pods -n monitoring
kubectl get pods -n argocd
kubectl get ingress -n db-stack
kubectl describe pod <name> -n db-stack
kubectl logs -n db-stack -l app=db-api --tail=50
```

15.3. Kubernetes — Application Operations

```
# Update name
source /vagrant/vagrant/scripts/load_env_var.sh
bash /vagrant/vagrant/scripts/update_name.sh "Alice"
```

```
# Restart a deployment
kubectl rollout restart deployment db-api -n db-stack
kubectl rollout restart deployment db-frontend -n db-stack
kubectl rollout status deployment db-api -n db-stack

# Scale the API (limited by topology spread to number of workers)
kubectl scale deployment db-api -n db-stack --replicas=2
```

15.4. Kubernetes — ArgoCD

```
# Get initial admin password
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d && echo

# Check sync status
kubectl get application -n argocd db-app

# Force immediate sync
kubectl -n argocd patch app db-app \
  -p '{"operation": {"sync": {}}}' --type merge
```

15.5. Kubernetes — Port Forwarding

```
kubectl port-forward svc/db-api -n db-stack 18000:80
kubectl port-forward svc/db-frontend -n db-stack 18080:80
kubectl port-forward svc/db-prometheus -n monitoring 9090:9090
kubectl port-forward svc/db-grafana -n monitoring 3000:3000
```

15.6. Build and Deploy

```
./build.sh                          # Build images locally
export REGISTRY=ghcr.io/dabeastnet
./push.sh                           # Push to registry
./deploy-k8s.sh                     # Apply all K8s manifests
./deploy-local.sh                   # Start Docker Compose
./test-local.sh                     # Smoke test Docker Compose
./test-k8s.sh                       # Smoke test Kubernetes
./package.sh                        # Create db-k8s-stack.zip
```

15.7. Testing API Endpoints

```
# Via ingress (Kubernetes)
curl -s http://localhost:18080/api/name
curl -s http://localhost:18080/api/container-id
curl -s http://localhost:18080/healthz
curl -s http://localhost:18080/readyz

# Direct (Docker Compose)
curl -s http://localhost:18000/api/name
curl -s http://localhost:18000/metrics | head -5
```

15.8. Verifying Load Balancing (PowerShell)

```
1..6 | ForEach-Object {
    (curl "http://localhost:18080/api/container-id") -match '"hostn
ame":"([^"]*)"' | Out-Null
    $matches[0]
}
```

16. Appendix C: Glossary

Term	Definition
ArgoCD	GitOps continuous delivery tool for Kubernetes; watches a Git repository and applies changes automatically
Alembic	Python database migration framework, used by SQLAlchemy projects
ClusterIP	Kubernetes service type exposing a stable internal IP; not reachable from outside the cluster
CNI	Container Network Interface — plugin providing pod networking. This project uses Flannel
containerd	Industry-standard container runtime used by kubeadm clusters
DaemonSet	Kubernetes workload that runs exactly one pod on every (matching) node
Downward API	Kubernetes mechanism to expose pod/node metadata to containers as environment variables
Flannel	Simple overlay network CNI using VXLAN encapsulation
GitOps	Operational model where the desired cluster state is stored in Git; an operator reconciles the live state
hostPath PV	Kubernetes PersistentVolume backed by a directory on the node's filesystem
ingress-nginx	nginx-based Kubernetes Ingress controller; routes HTTP traffic based on Host header and path
kubeadm	Official tool for bootstrapping a Kubernetes cluster
kube-state-metrics	Exposes Kubernetes object state (pod phases, conditions) as Prometheus metrics

NodePort	Kubernetes service type exposing a port on every cluster node; used when no cloud load-balancer is available
node-exporter	Prometheus exporter for hardware and OS metrics (CPU, memory, disk, network)
PVC / PV	PersistentVolumeClaim / PersistentVolume — Kubernetes storage abstraction
Readiness probe	Kubernetes probe that determines whether a pod should receive traffic
Liveness probe	Kubernetes probe that determines whether a pod should be restarted
Startup probe	Kubernetes probe evaluated only at container startup; delays liveness/readiness evaluation
StatefulSet	Kubernetes workload for stateful applications; provides stable network identities and ordered scaling
Topology spread	Kubernetes scheduling constraint distributing pods evenly across nodes/zones
Uvicorn	ASGI server used to run FastAPI applications
VXLAN	Virtual Extensible LAN — network encapsulation protocol used by Flannel for pod-to-pod communication
Vagrant	Tool for managing reproducible VM environments

17. Appendix D: Python Dependencies

Package	Version	Purpose
fastapi	0.111.0	Web framework
uvicorn[standard]	0.23.2	ASGI server
SQLAlchemy	2.0.29	ORM and connection pooling
alembic	1.13.1	Database migration tool
psycopg2-binary	2.9.9	PostgreSQL adapter (binary distribution, no compilation needed)
python-dotenv	1.0.1	`.env` file loading for local development
prometheus-client	0.20.0	Prometheus metrics exposition

18. Appendix E: URLs & Endpoints Reference

18.1. Public URLs (via Cloudflare Tunnel)

URL	Service	Notes
`https://project.beckersd.com/`	Frontend (Apache)	Serves `index.html`
`https://project.beckersd.com/api/name`	db-api	Returns `{"name": "..."}` from PostgreSQL
`https://project.beckersd.com/api/container-id`	db-api	Returns container ID, pod name, hostname
`https://project.beckersd.com/healthz`	db-api	Liveness probe — `{"status": "ok"}`
`https://project.beckersd.com/readyz`	db-api	Readiness probe — HTTP 200 or 503 if DB down
`https://project.beckersd.com/metrics`	db-api	Prometheus metrics (text/plain)
`https://argocd.beckersd.com`	ArgoCD UI	GitOps dashboard — login: `admin` / see secret

18.2. Local Access (Vagrant Port Forwarding)

Host URL	Host Port	NodePort (cp1)	Service
`http://localhost:18080/`	18080	30080	nginx ingress → frontend
`http://localhost:18080/api/name`	18080	30080	nginx ingress → db-api
`http://localhost:18080/api/container-id`	18080	30080	nginx ingress → db-api
`http://localhost:18080/healthz`	18080	30080	nginx ingress → db-api
`http://localhost:18080/readyz`	18080	30080	nginx ingress → db-api
`http://localhost:18080/metrics`	18080	30080	nginx ingress → db-api
`https://localhost:18443/`	18443	30443	nginx ingress (TLS)
`http://localhost:19090/`	19090	30090	Prometheus UI
`http://localhost:13000/`	13000	30030	Grafana UI — login: `admin` / `admin`

18.3. NodePorts (direct on cp1: 192.168.56.10)

URL	NodePort	Service
`http://192.168.56.10:30080/`	30080	nginx ingress HTTP
`https://192.168.56.10:30443/`	30443	nginx ingress HTTPS
`http://192.168.56.10:30090/`	30090	Prometheus UI
`http://192.168.56.10:30030/`	30030	Grafana UI

18.4. API Endpoints

All endpoints are served by FastAPI on container port `8000`, exposed via the `db-api` ClusterIP service on port `80`.

Method	Path	Description	Response
`GET`	`/api/name`	Name from PostgreSQL `person` table	`{"name": "Dieter Beckers"}`
`GET`	`/api/container-id`	Container identity	`{"container_id": "abc123", "pod_name": "...", "hostname": "worker1"}`
`GET`	`/healthz`	Liveness probe	`{"status": "ok"}` — HTTP 200
`GET`	`/readyz`	Readiness probe (checks DB)	`{"status": "ok"}` — HTTP 200, or HTTP 503

`GET`	`/metrics`	Prometheus metrics	`text/plain` — `db_api_requests_total` counter

18.5. Internal Cluster DNS

DNS name	Port	Service
`db-api.db-stack.svc.cluster.local`	80	API (FastAPI)
`db-frontend.db-stack.svc.cluster.local`	80	Frontend (Apache)
`db-postgres.db-stack.svc.cluster.local`	5432	PostgreSQL
`db-prometheus.monitoring.svc.cluster.local`	9090	Prometheus
`db-grafana.monitoring.svc.cluster.local`	3000	Grafana
`db-kube-state-metrics.monitoring.svc.cluster.local`	8080	kube-state-metrics
`db-ingress-nginx-controller.ingress-nginx.svc.cluster.local`	80 / 443	nginx ingress (Cloudflare tunnel target)

18.6. Prometheus Scrape Targets

Job	Target	Path	What it scrapes
`prometheus`	`localhost:9090`	`/metrics`	Prometheus itself
`kube-state-metrics`	`db-kube-state-metrics.monitoring.svc.cluster.local:8080`	`/metrics`	Kubernetes object state

`db-api`	`db-api.db-stack.svc.cluster.local:80`	`/metrics`	Application request counters
`kubernetes-nodes`	Each node IP `:9100` (auto-discovered)	`/metrics`	node-exporter: CPU, memory, disk, network

---

19. Appendix F: Cloudflare Tunnel Setup

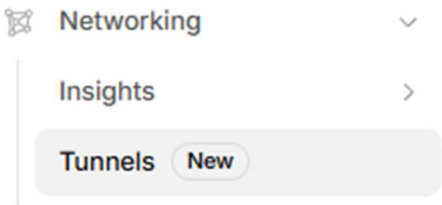
The cluster has no public IP and no open inbound firewall ports. Instead, a `cloudflared` pod inside the cluster opens an outbound tunnel to Cloudflare's edge. All traffic to `project.beckersd.com` and `argocd.beckersd.com` arrives through this tunnel — no LoadBalancer or NodePort exposure to the internet is required.

19.1. Prerequisites

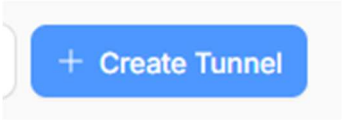
- A Cloudflare account with a zone (domain) managed by Cloudflare DNS.
- The domain `beckersd.com` already added to Cloudflare (free plan is sufficient).

19.2. Create the Tunnel

1. Log in to the Cloudflare dashboard at **https://one.dash.cloudflare.com**.
2. Navigate to **Networking > Tunnels**.

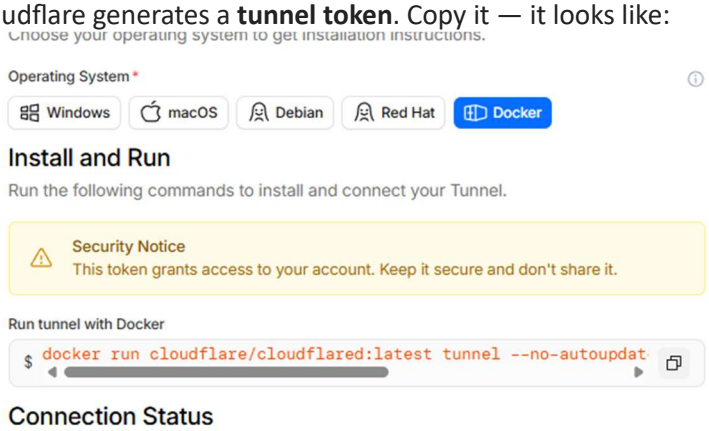


3. Click **Create a tunnel**.



4. Give the tunnel a name (e.g., `ProjectTunnel`) and click **create tunnel**.

5. Cloudflare generates a **tunnel token**. Copy it — it looks like:



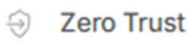

```
docker run cloudflare/cloudflared:latest tunnel --no-autoupdate run
--token eyJhIjoiMj…pKaCJ9
```

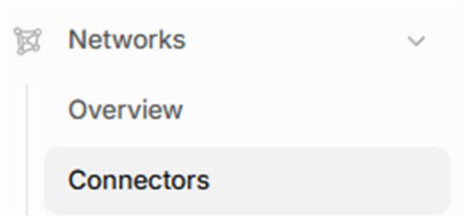
1. Paste the token into `k8s/cloudflared/deployment.yaml` under `stringData.token`:

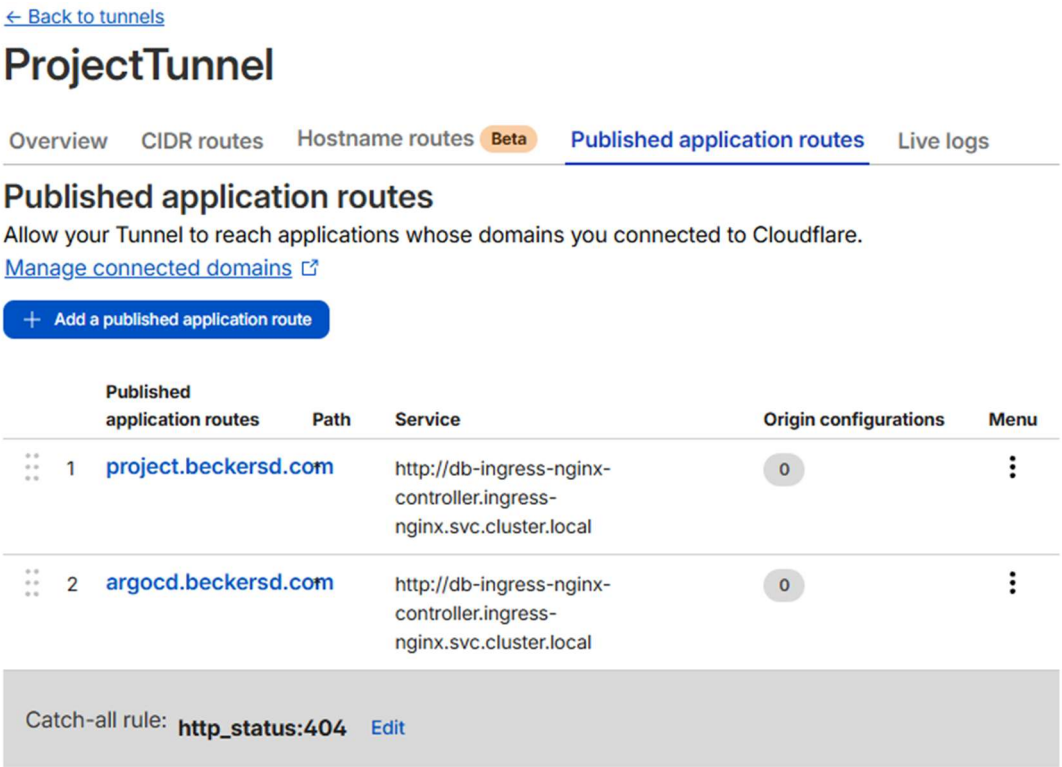
```
stringData:
token: "eyJhIjoiMj…pKaCJ9"
```

19.3. Add Public Hostnames (DNS entries)

After saving the tunnel, go to the **Zero Trust > Networks>Connectors>Your Tunnel>Published application routes** tab and add two entries:







Subdomain	Domain	Type	Service
`project`	`beckersd.com`	HTTP	http://db-ingress-nginx-controller.ingress-nginx.svc.cluster.local
`argocd`	`beckersd.com`	HTTP	http://db-ingress-nginx-controller.ingress-nginx.svc.cluster.local

Note: Both entries point to the same nginx ingress NodePort (`30080`). The ingress controller uses the `Host` header to route:

- `project.beckersd.com` → `db-frontend` and `/api` → `db-api`
- `argocd.beckersd.com` → `db-argocd-server`

Cloudflare automatically creates `CNAME` DNS records for both hostnames pointing to `<tunnel-id>.cfargotunnel.com`.

19.4. TLS

Cloudflare terminates TLS at the edge for free using its Universal SSL certificate (`*.beckersd.com`). The tunnel carries plain HTTP between Cloudflare and the cluster — no certificate configuration is needed inside the cluster.

Important: The wildcard cert `*.beckersd.com` covers single-level subdomains only (e.g., `project.beckersd.com`). It does **not** cover two levels deep (e.g., `project.app.beckersd.com`). This is why `argocd.beckersd.com` is used instead of something like `argocd.project.beckersd.com`.

19.5. Deploy the Tunnel Pod

Once the token is set in the manifest, apply it to the cluster:

```
kubectl apply -f k8s/cloudflared/deployment.yaml
kubectl rollout status deployment/db-cloudflared -n db-stack
```

Verify the tunnel is connected in the Cloudflare dashboard — the tunnel status should show **Healthy** with one connector.

19.6. Verify End-to-End

```
curl https://project.beckersd.com/api/name
# {"name":"Dieter Beckers"}

curl https://project.beckersd.com/healthz
# {"status":"ok"}
```